# Mathematical modelling of T-Adaptive Segment Routing

Project Orange – Challenge EURO/ROADEF 2026, IT4033E Capstone

Nguyễn Tiến Đức – 20252076M  Hoàng Huy Chiến – 20261069M

October 2026

## 1 Introduction

**Context.** Orange operates large IP/MPLS backbones carrying tens of thousands of traffic demands. Traffic engineering (TE) tunes the routing so that no link becomes congested, and the network must stay functional when links are taken down, either by failures or by planned maintenance *interventions*. Classical IP routing sends each packet along the shortest paths for fixed link weights; when several shortest paths exist, the Equal-Cost Multi-Path (ECMP) rule splits the flow between them. Changing weights to avoid congestion is hard and affects every demand at once.

**Motivation.** *Segment Routing* (SR) lets the operator attach to each demand a short list of *waypoints* that the packets must visit in order; between two consecutive waypoints the usual shortest-path/ECMP routing is used. Choosing waypoints therefore bends selected demands away from overloaded links without touching the weights. Each waypoint change is, however, an operational action, so the number of changes between two periods is limited.

**Problem.** The T-Adaptive Segment Routing problem (T-ASR) of the challenge is: given a network $(G = (V, A), \omega, c)$, demands $D$ with time-dependent volumes $\nu(d, t)$, time periods $T = \{0, \dots, h-1\}$, the set $q(t) \subseteq A$ of arcs down at period $t \geq 1$, a change budget $\kappa(t)$ and a maximum number of segments $maxSeg$, choose for every demand and period a segment path so that the link loads are as small as possible. Period 0 is the nominal network; in the instances of this project $h = 2$ and period 1 has some links down. Following the course project description, we first solve period 0 alone (no budget constraint), then periods 0 and 1 together with the budget constraint, on the 20 instances of set A, with the exact MILP of the subject solved by branch-and-bound.

## 2 Data

**Input files.** Each instance consists of three JSON files. `*-net.json` gives the vertices and the arcs $a = (i, j)$ with metric $\omega(a)$ and capacity $c(a)$; `*-tm.json` gives the demands $d = (s_d, t_d)$ and their volumes $\nu(d, 0), \nu(d, 1)$; `*-scenario.json` gives $maxSeg$, the budget $\kappa(1)$ and the links $q(1)$ that are down at period 1. The graph of period $t$ is $G(t) = (V, A \setminus q(t))$ with $q(0) = \emptyset$. Vertices and arcs are renumbered $1..n$ and $1..m$; several files declare a `"multigraph"` flag although they contain no parallel arcs, so we only check that no two arcs share the same ordered pair of end vertices.

**Split coefficients.** $r(i, j, a, t)$ is the fraction of a unit flow sent from $i$ to $j$ in $G(t)$ that crosses arc $a$. Following the course document *Computing arc-use ratios on shortest paths*, it is the fraction of shortest $i$–$j$ paths that use $a$:

$$r(i, j, a, t) = \frac{\sigma^a_{ij}(t)}{\sigma_{ij}(t)} = \begin{cases} \dfrac{\sigma_{iu}(t)\,\sigma_{vj}(t)}{\sigma_{ij}(t)} & \text{if } d_{iu}(t) + \omega(a) + d_{vj}(t) = d_{ij}(t),\ a = (u, v), \\ 0 & \text{otherwise,} \end{cases}$$

where $d_{ij}(t)$ is the shortest distance and $\sigma_{ij}(t)$ the number of shortest paths in $G(t)$. Both are obtained by one Dijkstra run per source that also counts paths ($\sigma(v) \leftarrow \sigma(u)$ on a strict improvement, $\sigma(v) \leftarrow \sigma(v) + \sigma(u)$ on a tie), i.e. $n$ runs per period, $O(n\,m \log n)$ in total. Only the non-zero coefficients are stored. The equality test uses a relative tolerance of $10^{-9}$. Note that this path-counting rule is not always equal to the per-node ECMP split of the subject: on the toy instance of the subject, $r(v_0, v_5, a_{1,3}, 0)$ is 1/3 with path counting (1 of the 3 shortest paths) and 1/2 with ECMP. We use the course definition throughout.

**Other parameters.** $c(a)$ from the net file, $\nu(d,t)$ from the tm file, $maxSeg$, $\kappa(1)$ and $q(1)$ from the scenario file. A segment $(i,j)$ can only be used at period $t$ if $j$ is reachable from $i$ in $G(t)$; we denote by $K \subseteq \{(i,j) : i \neq j\}$ the pairs reachable in at least one period.

# 3 Mathematical model

**Variables.**

- $x_{ij}^{dt} \in \{0,1\}$ for $d \in D$, $(i,j) \in K$, $t \in T$: equals 1 iff the segment path of demand $d$ at period $t$ contains the segment $(i,j)$, i.e. $j$ is visited right after $i$ (both being the source, a waypoint or the target). $x_{ij}^{dt}$ is fixed to 0 when $j$ is unreachable from $i$ in $G(t)$.
- $U \geq 0$: the maximum link utilization (MLU) over all available arcs and all periods.
- $y_{ij}^{dt} \geq 0$ for $t \geq 1$: auxiliary variables linearizing $|x_{ij}^{dt} - x_{ij}^{d,t-1}|$.

**Objective.**

$$\min\ U.$$

The subject asks for the lexicographic minimization of the sorted vector of all loads $\lambda(a,t)$. Its first and dominant criterion is the largest load, i.e. the MLU; we optimize this criterion only, which is the classical TE objective and keeps a single-objective MILP. (A full lexicographic optimization would fix $U$ at its optimum and re-optimize the next criterion, multiplying the solving effort.)

**Constraints.**

$$\sum_{j:(i,j)\in K} x_{ij}^{dt} - \sum_{j:(j,i)\in K} x_{ji}^{dt} = \begin{cases} 1 & i = s_d \\ -1 & i = t_d \\ 0 & \text{otherwise} \end{cases} \qquad \forall i \in V,\ d \in D,\ t \in T \qquad (1)$$

$$\sum_{(i,j)\in K} x_{ij}^{dt} \leq maxSeg \qquad \forall d \in D,\ t \in T \qquad (2)$$

$$\sum_{d\in D} \sum_{(i,j)\in K} r(i,j,a,t)\,\nu(d,t)\,x_{ij}^{dt} \leq U\,c(a) \qquad \forall t \in T,\ a \in A \setminus q(t) \qquad (3)$$

$$\sum_{d\in D} \sum_{(i,j)\in K} y_{ij}^{dt} \leq \kappa(t), \quad y_{ij}^{dt} \geq x_{ij}^{dt} - x_{ij}^{d,t-1}, \quad y_{ij}^{dt} \geq x_{ij}^{d,t-1} - x_{ij}^{dt} \qquad \forall t \in T^* \qquad (4)$$

- (1) *Flow conservation on the segment graph*: for each demand and period, the chosen segments form a path from $s_d$ to $t_d$ in the complete graph of segments: one more segment leaves the source than enters it, one more enters the target, and every other vertex is entered as often as it is left.
- (2) *Number of segments*: a segment path with $\ell$ waypoints has $\ell + 1$ segments; the router technology allows at most $maxSeg$ (= 6 in set A, so at most 5 waypoints).
- (3) *Load*: the traffic on arc $a$ is the sum, over demands and their chosen segments, of the demand volume times the fraction of the segment's ECMP flow that crosses $a$. Dividing by $c(a)$ gives the load $\lambda(a,t)$, so (3) states $\lambda(a,t) \leq U$; arcs that are down are excluded, and

since $r(\cdot,\cdot,a,t) = 0$ for them no flow is sent on them. Minimizing $U$ makes $U = \max_{a,t} \lambda(a,t)$ at the optimum.

- (4) *Budget*: the number of segments added or removed between periods $t-1$ and $t$, $\mathrm{dist}(P^{t-1}, P^t)$, is at most $\kappa(t)$. The absolute value is linearized with $y$; since $y$ only appears in the budget sum, any $x$ satisfying the original constraint can take $y = |x^t - x^{t-1}|$, so the linearization is exact. Only used in the two-period model.

**Remarks.** (i) Constraints (1)–(2) allow, besides the $s_d$–$t_d$ path, extra cycles of segments disjoint from it. Such cycles only add load, so an optimal solution never needs them, and we read the waypoints by walking from $s_d$. (ii) Routing every demand on its shortest paths ($x^{dt}_{s_d t_d} = 1$, no waypoint) satisfies (1)–(4) with zero changes, so the model is always feasible whenever each $t_d$ is reachable from $s_d$ in every $G(t)$. We give this solution to the solver as a MIP start and report its MLU as the *no-waypoint* value. (iii) Model size: $|D|\,|K|\,|T|$ binary variables with $|K| \le n(n-1)$, $|T|(|D|(n+1)+m)$ constraints for (1)–(3) and $2|D||K|+1$ for (4). The tables report the total number of variables.

# 4 Implementation

The model is written in Julia 1.13.2 with JuMP 1.32.1 and solved by HiGHS 1.15.1 (HiGHS.jl 1.26.0); graphs and shortest paths use Graphs.jl and the JSON files are read with JSON3.jl. Load constraints (3) are assembled arc by arc from the sparse coefficients ($r > 0$ only). The code is checked on the toy instance of the subject (24 automated tests: split coefficients, loads, and equality of the MILP optimum with a brute-force enumeration of all segment paths for periods $\{0\}$ and $\{0,1\}$; optimal MLU 0.375 with waypoint $v_4$ for demand $(v_0, v_5)$). The *CPU time* reported is the time of the `optimize!` call only; the model-building time is recorded separately.

# 5 Numerical experiments

**Instances.** Set A contains 20 instances with $20 \le n \le 400$ vertices, $80 \le m \le 2000$ arcs, $20 \le |D| \le 6000$ demands, $maxSeg = 6$ and $h = 2$ periods (columns $n, m, |D|$ of Tables 1–2). The number of binary variables grows as $|D|\,n^2$, from $1.5 \cdot 10^4$ (instance 01) to almost $10^9$ (instance 20).

**Running environment.** All reported runs were executed on one machine: Intel Core i9-14900K, 24 logical CPUs available, 125 GB RAM, Ubuntu 24.04 (Linux 7.0). Each run uses HiGHS with one thread; several independent runs are executed in parallel processes (at most 16 at once). The machine was shared with other users, which adds some noise to the CPU times.

**Protocol.** Each instance is solved ten times with HiGHS random seeds $1, \ldots, 10$; the tables give the average over the ten runs. HiGHS default settings are used, except presolve, which is switched off for the two-period model (see the comments below). Time limit: 1800 s for period 0 (Table 1) and 3600 s for periods 0 and 1 (Table 2). Columns: number of variables; number of runs that proved optimality; average best objective value (MLU); MLU of the no-waypoint MIP start; average gap $1 - \mathrm{LB}/\mathrm{UB}$ (0 when optimal; when HiGHS stops without any bound we use the trivial bound $\mathrm{LB} = 0$ given by $U \ge 0$, i.e. a gap of 100%); average CPU time of `optimize!`; average number of branch-and-bound nodes (0 means the solver stopped before finishing the root node). An instance whose model needs more memory than available is reported as *model too large*.

**Comments.**

- *Small instances (01–03)* are solved to optimality at the root node in seconds for period 0. Waypoints lower the MLU from 1.00 (no waypoint) to 0.93, 0.55 and 0.94: on instance 02 the optimal routing almost halves the most loaded link.

Table 1: Period 0 only (no budget constraint), time limit 1800 s, averages over 10 runs.

| Inst | $n$ | $m$ | $\|D\|$ | vars | opt/runs | MLU | no-wp | gap (%) | CPU (s) | nodes |
|---|---|---|---|---|---|---|---|---|---|---|
| 01 | 20 | 80 | 40 | 0.02M | 10/10 | 0.929 | 1.000 | 0.0 | 4.0 | 1.0 |
| 02 | 30 | 150 | 45 | 0.04M | 10/10 | 0.549 | 1.000 | 0.0 | 27.8 | 1.0 |
| 03 | 50 | 250 | 20 | 0.05M | 10/10 | 0.944 | 1.000 | 0.0 | 16.3 | 1.0 |
| 04 | 50 | 250 | 200 | 0.49M | 0/10 | 0.675 | 0.692 | 31.5 | 1819.3 | 0.0 |
| 05 | 100 | 396 | 100 | 0.99M | 4/10 | 0.135 | 0.333 | 12.6 | 1578.5 | 0.4 |
| 06 | 100 | 500 | 500 | 4.95M | 0/10 | 1.000 | 1.000 | 100.0 | 2056.4 | 0.0 |
| 07 | 100 | 500 | 800 | 7.92M | 2/10 | 0.908 | 0.908 | 80.0 | 2069.4 | 0.2 |
| 08 | 150 | 654 | 200 | 4.47M | 1/10 | 0.800 | 1.000 | 53.8 | 12450.4 | 0.1 |
| 09 | 150 | 750 | 200 | 4.47M | 0/10 | 0.611 | 0.611 | 17.3 | 2403.3 | 0.0 |
| 10 | 150 | 966 | 1000 | 22.35M | 0/10 | 1.000 | 1.000 | 100.0 | 2989.6 | 0.0 |
| 11 | 200 | 1000 | 400 | 15.92M | 0/10 | 1.000 | 1.000 | 100.0 | 2430.3 | 0.0 |
| 12 | 200 | 898 | 400 | 15.92M | 0/10 | 1.000 | 1.000 | 100.0 | 2342.9 | 0.0 |
| 13 | 200 | 1000 | 2000 | 79.60M | | | model too large | | | |
| 14 | 250 | 1108 | 600 | 37.35M | 0/10 | 1.000 | 1.000 | 100.0 | 3032.5 | 0.0 |
| 15 | 250 | 1250 | 600 | 37.35M | 0/10 | 1.000 | 1.000 | 100.0 | 2826.6 | 0.0 |
| 16 | 250 | 1452 | 4800 | 298.80M | | | model too large | | | |
| 17 | 300 | 1270 | 2000 | 179.40M | | | model too large | | | |
| 18 | 300 | 1500 | 2000 | 179.40M | | | model too large | | | |
| 19 | 300 | 1998 | 6000 | 538.20M | | | model too large | | | |
| 20 | 400 | 2000 | 6000 | 957.60M | | | model too large | | | |

- *Medium instances (04–05)* are not proved optimal within 30 minutes; the solver spends the time in the root node (cuts and heuristics, 0 nodes). The result depends on the random seed: on instance 05, 4 of the 10 seeds reach and prove the optimum 0.1155 (gap 0) while the others stop with gaps up to 30%; on instance 04 only 1 seed of 10 improves the MIP start ($0.692 \to 0.522$). This variability justifies averaging over ten runs.
- *Large instances (06–12)* have millions of binary variables. The LP relaxation alone takes most of the time limit, and the solver usually returns the MIP start. On some of them the no-waypoint MLU is already optimal (e.g. instance 07: proved optimal at 0.908 by two seeds): when the most loaded arc is crossed by every possible route of a large demand, no waypoint can relieve it. On instance 08 the seed matters most: one seed proves the optimum 0.319 in 1744 s, three others stop at 0.561 and the rest keep the MIP start (1.000). The time limit is not always respected on these models (probably because HiGHS checks it only between algorithmic steps): runs on instance 08 stopped only after 10409 s, 37322 s and 51940 s, which inflates the average CPU time (average 12450 s). On instance 10 ($2.2 \cdot 10^7$ variables) all ten runs stop between 2620 s and 3449 s (limit 1800 s) with the MIP start (1.000) and no lower bound, i.e. gap 100%; all ten runs of instances 11 and 12 ($1.6 \cdot 10^7$ variables each) behave the same way (2313–2485 s and 2293–2511 s). Instances 14 and 15 ($3.7 \cdot 10^7$ variables each, the largest models we could build) give the same picture for all their runs (3019–3048 s and 2713–3026 s).
- *Two periods.* Adding period 1 triples the number of variables (two copies of $x$ plus $y$). With default settings, HiGHS presolve stalls on the budget row (4), a single dense row over $|D||K|$ variables: on instance 04 it was still in presolve after more than three hours with a one-hour limit. Removing (4) or switching presolve off restores the time limit, so all runs of Table 2 use presolve off. Table 3 gives the runs made before this change on the small instances, with presolve on. On instance 02 the link down at period 1 makes the no-waypoint MLU 1.098 (congestion); with presolve on only 2 of the 10 seeds find and prove the optimum 0.903 (in 114 s and 3307 s), the others stop at the time limit with MLU between 1.043 and 1.098, which shows how much the solver's search path matters on this instance. With presolve off, 7 of the 10 seeds prove the same optimum 0.903 (instance 03: 3 of 10, against 4 of 10 with presolve on).

Table 2: Periods 0 and 1 with budget constraint, HiGHS presolve off, time limit 3600 s, averages over 10 runs.

| Inst | $n$ | $m$ | $\|D\|$ | vars | opt/runs | MLU | no-wp | gap (%) | CPU (s) | nodes |
|---|---|---|---|---|---|---|---|---|---|---|
| 01 | 20 | 80 | 40 | 0.05M | 10/10 | 0.929 | 1.000 | 0.0 | 23.4 | 11.0 |
| 02 | 30 | 150 | 45 | 0.12M | 7/10 | 0.942 | 1.098 | 5.2 | 2386.5 | 6.1 |
| 03 | 50 | 250 | 20 | 0.15M | 3/10 | 0.961 | 1.000 | 24.0 | 3109.1 | 44.4 |
| 04 | 50 | 250 | 200 | 1.47M | 0/10 | 0.898 | 1.000 | 32.9 | 3638.6 | 0.0 |
| 05 | 100 | 396 | 100 | 2.97M | 0/10 | 0.865 | 1.000 | 77.1 | 5198.7 | 0.0 |
| 06 | 100 | 500 | 500 | 14.85M | 0/5 | 1.000 | 1.000 | 100.0 | 3893.4 | 0.0 |
| 08 | 150 | 654 | 200 | 13.41M | 0/1 | 1.000 | 1.000 | 68.1 | 3892.2 | 0.0 |
| 09 | 150 | 750 | 200 | 13.41M | 0/5 | 1.000 | 1.000 | 66.0 | 3869.9 | 0.0 |
| 10 | 150 | 966 | 1000 | 67.05M | model too large | | | | | |
| 11 | 200 | 1000 | 400 | 47.76M | model too large | | | | | |
| 12 | 200 | 898 | 400 | 47.76M | model too large | | | | | |
| 13 | 200 | 1000 | 2000 | 238.80M | model too large | | | | | |
| 14 | 250 | 1108 | 600 | 112.05M | model too large | | | | | |
| 15 | 250 | 1250 | 600 | 112.05M | model too large | | | | | |
| 16 | 250 | 1452 | 4800 | 896.40M | model too large | | | | | |
| 17 | 300 | 1270 | 2000 | 538.20M | model too large | | | | | |
| 18 | 300 | 1500 | 2000 | 538.20M | model too large | | | | | |
| 19 | 300 | 1998 | 6000 | 1614.60M | model too large | | | | | |
| 20 | 400 | 2000 | 6000 | 2872.80M | model too large | | | | | |

Table 3: Periods 0 and 1 with HiGHS presolve on (default), time limit 3600 s, averages over 10 runs.

| Inst | $n$ | $m$ | $\|D\|$ | vars | opt/runs | MLU | no-wp | gap (%) | CPU (s) | nodes |
|---|---|---|---|---|---|---|---|---|---|---|
| 01 | 20 | 80 | 40 | 0.05M | 10/10 | 0.929 | 1.000 | 0.0 | 32.6 | 1.0 |
| 02 | 30 | 150 | 45 | 0.12M | 2/10 | 1.053 | 1.098 | 42.9 | 3224.8 | 0.7 |
| 03 | 50 | 250 | 20 | 0.15M | 4/10 | 0.946 | 1.000 | 11.5 | 2840.8 | 0.8 |

On instance 04 no seed proves optimality but every run improves the MIP start (1.000 → 0.898 on average, best 0.645, bound 0.581). On instance 05, 2 of the 10 seeds find 0.323 (no-waypoint 1.000, bound 0.205), the other 8 keep the MIP start; runs take between 3684 s and 10049 s for a 3600 s limit. On instance 06 ($1.5 \cdot 10^7$ variables) the five runs finished so far keep the MIP start (1.000) without any lower bound (gap 100%), in 3875–3945 s. Instances 09 (5 runs) and 08 (1 run so far) also keep the MIP start; some seeds reach a lower bound (0.850 on 09, 0.319 on 08), so the gap is not always 100%.

- *Very large instances (13–20)* need between $3.7 \cdot 10^7$ and $9.6 \cdot 10^8$ binary variables for a single period, and our implementation uses about 2 GB of memory per million variables. Only 14 and 15 ($3.7 \cdot 10^7$ variables, ≈ 75 GB) fit in the 125 GB of the test machine; the others cannot be built. For the two-period model with presolve off, memory is 3.7 to 5.5 GB per million variables (instance 06: $1.5 \cdot 10^7$ variables, 54.8 GB; instance 05: $3.0 \cdot 10^6$, 16 GB). The two-period model of 07 ($2.4 \cdot 10^7$ variables, ≈ 88 GB) fits only when run alone, and those of 10–20 ($\geq 4.8 \cdot 10^7$) do not fit. Solving them would require a decomposition (e.g. column generation over segment paths) or a heuristic restricting the candidate waypoints.

## 6 Experimental process: problems met and how they were solved

Obtaining the tables above was an iterative, trial-and-error process. Each obstacle was handled the same way: observe a symptom in the logs, form a hypothesis, test it on a small controlled experiment, then change one thing and re-run. Table 4 lists these steps in the order they occurred; several of them changed the experimental protocol, which is why we report them.

Table 4: Problems met during the experiments, diagnosis and resolution (chronological).

| Observation | Diagnosis (how it was verified) | Resolution / consequence |
|---|---|---|
| Toy instance: our $r(v_0, v_5, a_{1,3}, 0) = 1/3$, subject Table 2 gives $1/2$. | Path counting (course document) and per-node ECMP split differ when shortest paths re-join; checked by hand on the toy graph. | Keep the course definition; unit tests encode it; difference stated in Section 2. |
| Instance 05 rejected by the loader ("multigraph"). | 10 set-A files declare `"multigraph": true` but contain no parallel arcs (checked on all files). | Only real parallel arcs are rejected. |
| Instance 04 (period 0): no feasible solution after 300 s, 0 nodes. | HiGHS spends the whole time in the root node (cuts, heuristics) without finding a feasible point. | No-waypoint routing given as MIP start (always feasible); it also gives the reference column "no-wp". |
| Laptop (24 GB) cannot build instances above $\approx 5 \cdot 10^6$ variables. | Peak memory measured: $\approx 3$ GB per million variables on small models, $\approx 2$ GB at $8 \cdot 10^6$ (instance 07). | All official runs moved to a 125 GB server; a memory-aware job queue starts a run only when its estimated peak memory is free (shared machine). |
| Some runs exceed the time limit (instance 09: 4326 s, 08: 10409 s, 37322 s and 51940 s, 10: up to 3449 s for a 1800 s limit; two-period 05: up to 10049 s for 3600 s). | Likely cause: HiGHS checks the limit only between algorithmic steps, so a long step on a huge LP is not interrupted. Not confirmed: profiling (`perf`, `gdb`) is disabled for users on the shared server. | Reported as measured (CPU column is the real `optimize!` time). Since one instance-08 job took 15.6 h for 4 runs, the queue was reordered to finish the period-0 table (14, 15) before the two-period models of 06, 08, 09. |
| Two-period instance 04: model building took 5428 s (period 0 alone: 2 s). | Timing each part of the build: `sum(y)` over a JuMP `DenseAxisArray` is quadratic (4× more terms → 13× slower in an isolated test). | Generator form of the sum: build time 1.4 s. Build time is excluded from the reported CPU time anyway. |
| Two-period instance 04: `optimize!` still running after 3 h with a 1 h limit, no bound. | Ablation on a 60 s limit: without constraint (4), or with presolve off, the limit is respected (76 s); with both, it is not. HiGHS presolve stalls on the dense budget row. | All two-period runs use presolve off (Table 2); earlier presolve-on runs of 01–03 kept for comparison (Table 3); stalled runs discarded. |
| Runs stopped with bound $-\infty$ give an undefined gap. | The solver stopped before solving the root LP. | $U \geq 0$ is a valid bound: LB = 0, gap = 100%. |
| Two-period instance 05 with presolve off: 16 GB for $3 \cdot 10^6$ variables. | Without presolve the LP is not reduced: $\approx 5.5$ GB per million variables. | Two-period models of 07 and 10–20 first declared too large. |
| Two-period instance 06: peak 54.8 GB for $1.5 \cdot 10^7$ variables, below the 82 GB planned. | Peak memory (`VmHWM`) measured on the running job: 3.7 GB per million variables on 06, but 5.3 on 08 (71 GB for $1.3 \cdot 10^7$ variables): the rate depends on the instance, so each estimate was corrected as soon as it was measured. Size rows also counted only $x$, not $y$ (fixed: all variables). | Memory estimates lowered: two two-period jobs run in parallel; 07 (88 GB) queued alone; 10–20 remain too large. |

**Lessons.** (i) Validating the model on a toy instance against brute force before scaling up made it possible to separate modelling errors from solver behaviour: all later problems were computational, not in the model. (ii) Solver defaults are not neutral: presolve, normally beneficial, made the two-period model unsolvable, so the solver setting is part of the protocol and must be reported. (iii) For exact MILP models of this size, memory, not CPU time, is the first limit; measuring it early is what decided which instances could be run at all.

# 7 Conclusion

We formulated T-ASR as a compact MILP whose variables select segments, computed the split coefficients by shortest-path counting, and solved the model with HiGHS on set A. The exact approach solves the small instances to optimality quickly and shows that well-chosen waypoints

can reduce the MLU substantially, but the number of variables, $|D|\,n^2|T|$, makes it impractical beyond a few hundred thousand variables within the time limit, and impossible to build for the largest instances. Natural next steps are restricting $K$ to promising waypoints, column generation on segment paths, and the full lexicographic objective.

## References


[1] A. Benhamiche, Y. Carlinet, M. Chopin, E. Gourdin, N. Perrot. *Challenge EURO/ROADEF 2026 – Problem Description*, Orange Research, 2026.

[2] Course document *Computing arc-use ratios on shortest paths* (`calculating_r.pdf`), IT4033E, SoICT, 2026.

[3] Q. Huangfu, J. A. J. Hall. Parallelizing the dual revised simplex method. *Mathematical Programming Computation* 10:119–142, 2018.